

Open in app ↗

 Medium

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Advanced Graphics Programming in Unreal, part 2

11 min read · Jun 24, 2025



William Mishra-Manning



Listen



Share

... More

This is a seven-part article series.

Part 1: [Introduction](#)

Part 2: Rendering Abstractions (RHI, RDG) and Threading

Part 3: [Scenes, Views, and SceneViewExtension](#)

Part 4: [Global Shaders](#)

Part 5: [Simple Material Shaders](#)

Part 6: [Advanced Material Shaders](#)

Part 7: [Mesh-Material Shaders and Mesh Passes](#)

Threading, and Rendering Abstractions

When doing graphics programming in Unreal, it helps to know how the different rendering abstractions are layered on top of each other. For example, what are RHI command lists and how do they relate to RDG instances? Aren't they a bit redundant? What threads can you use an RHI command list on? And what's with the "immediate" command lists?

I'll go over the core abstraction layers at a high level so we won't feel disoriented when dealing with them later. You may also reference [the official Unreal docs](#) for

more details on the RDG layer.

I will also go over the multithreading design patterns in Unreal's renderer. [The official docs](#) have more details for you, but I have attempted to boil this down as much as possible to what's essential.

Theory

Threading

Unreal rendering is highly threaded. Top-level render logic happens synchronously, on the special "Render Thread" (or "RT" for short). This is usually the only thread you'll directly interact with. Intermediary work gets distributed among worker threads, and then the raw graphics API calls are executed on a special "RHI thread" which may actually be multiple threads if Unreal is using a modern API like Vulkan.

When debugging graphics stuff, the secondary threads can all be disabled through config files so that everything is synchronous on the Render Thread. See the first article in this series for how to modify config files for graphics debugging.

You can queue up code to run on the RT, from *anywhere* (including the RT itself), by calling:

```
ENQUEUE_RENDER_COMMAND(MyCommandsNameForDebugging)(myLambda)
```

The lambda takes one argument, of type `FRHICommandList` (more on that later).

It's very important for your lambda to capture most of its data by copy, because there can be frames of lag between when a command is queued up and when it executes. Any local variables it captured are certainly cleaned up and gone by the time it runs! Even the owning `UObject` may have been garbage-collected before the command executes (ways to prevent this are explained below).

While you can set up draw calls and compute shaders to run from

`ENQUEUE_RENDER_COMMAND`, they could only operate on off-screen data so it's more

commonly used to just pass data into the Render Thread.

In practice most rendering work will be within Scene View Extensions, covered in the next article.

Proxies

Multithreading bugs are a nightmare, so great pains are taken to separate game-thread logic (like `AActor`, `UActorComponent`, and `UWorld`) from render-thread logic. You must understand and follow these conventions in your own graphics code.

Every object that participates in rendering (mesh, material, component, world, etc) has a corresponding *proxy* struct that is not `UObject`, not garbage-collected, and contains very simple data. In fact it's usually completely disconnected from its owning `UObject`. The render thread should only concern itself with proxies and not the original objects, to prevent race conditions — especially in case the original `UObject` gets destroyed! Any object that has a presence in the Render Thread will usually do the following:

1. Store its proxy as a private field
2. Offer a public getter for the proxy that checks `IsInRenderThread()`
3. Periodically update the proxy with `ENQUEUE_RENDER_COMMAND()`
4. Set up an `FRenderCommandFence` when dying to ensure the proxy never gets used after death (see below for more details).

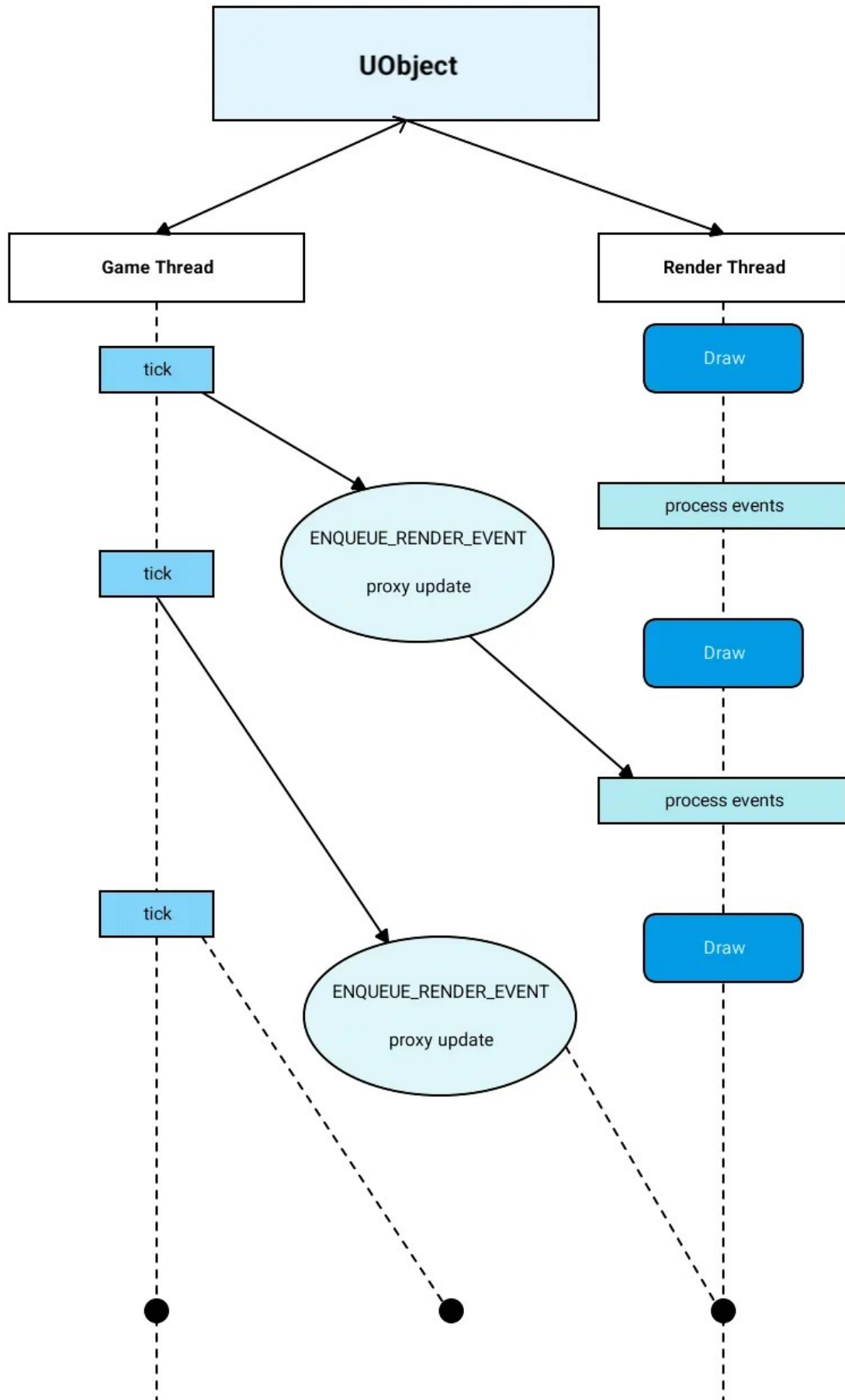
Many proxies keep a Debug pointer to their owning game object for convenience.

It's important to understand what purpose a C++ lambda serves in

`ENQUEUE_RENDER_COMMAND()`. Your new proxy data should be captured and owned by the lambda, safely carrying it outside the lifetime of the calling function and into the rendering thread a frame or two in the future. Then during execution of the lambda, that captured copy can be moved into a more persistent render-thread-only variable which usually belongs to the `UObject` or its proxy.

A note about performance: in the above workflow, proxy data is copied several times per

shift to the Render-Thread. If it has a heap-allocated collection like `TArray`, then use `MoveTemp()` to avoid copies and ideally use an inline allocator to avoid heap usage altogether.



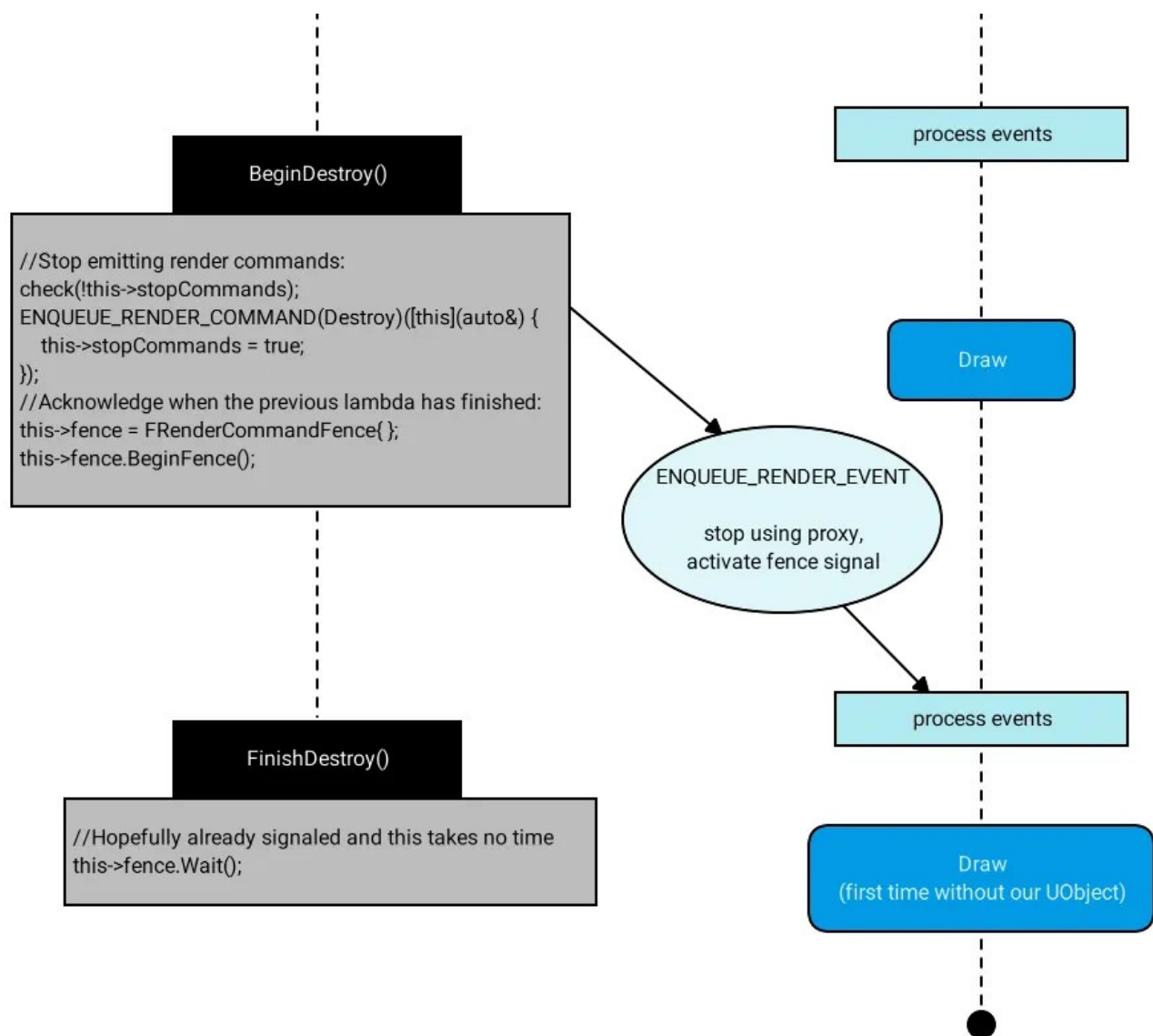


Diagram of a `UObject`'s render logic. The Destroy logic is explained below; 'stopCommands' is a stand-in for some kind of signal to your rendering code.

While it's generally not safe to access a `UObject` from other threads without extra work you *can* always use its pointer, for example as a lookup key. After all, pointers are just integers with a fancy coat of paint (don't tell a C++ expert I said that).

UObjects on the Render Thread

In practice you do often need to dereference certain `UObjects` on the Render Thread, for example any object that holds its own proxy. In those cases you need to make sure the `UObject` doesn't get cleaned up by the GC (or killed, if it's an actor/component) until the Render Thread knows to stop referencing it. [Here is an external article about multithreaded cleanup.](#)

It's recommended to use an `FRenderCommandFence`, for the Render Thread to acknowledge that the object is dying and then for the Game Thread to wait for that acknowledgement before finishing destruction. Refer to my `UObject` diagram above. In fact this is exactly why objects have separate `BeginDestroy()` and `FinishDestroy()` events with a bit of lag between them — start the fence on `Begin` and wait for the fence on `Destroy`.

*If you're thinking of using simple mutex locks, please don't! It ruins the whole point of a multithreaded architecture, by forcing it to act single-threaded. The only acceptable uses for locking the render/game thread are in the editor or for special **infrequent** operations.*

Code Organization

Outside of normal game logic, functions and variables should make it extremely clear when they're meant to be called on a specific thread. If the function is for the render thread, suffix it with `_RenderThread` and add `check(IsInRenderingThread())` on the first line. Similarly, for the game thread use `_GameThread` and `check(IsInGameThread())`. It's not uncommon to have two versions of a variable or function, with each prefix.

Unfortunately not all rendering functions follow this convention, and you'll have to check their documentation or source before calling them. It might be implicit based on their arguments, for example if they take an `FRHICommandList` or `FRDGBuilder` then they're almost certainly meant for the render thread.

Rendering Abstractions

RHI

The term “RHI” stands for “Render Hardware Interface” and covers a specific graphics API such as DirectX12 or Vulkan. Each RHI also has a reported “feature level” indicating what broad capabilities it has. As of Unreal 5.3 the only meaningful feature levels are `SM5` for older hardware, `SM6` for newer hardware, and `ES3_1` for mobile hardware.



You can think of low-level graphics calls as being split into three layers. From lowest to highest:

- RHI global function calls
- RHI Command Lists
- Render Dependency Graphs

Global RHI functions

TL;DR: In practice there is very little need to use this layer directly, except for a handful of commands and possibly for debugging/prototyping. Instead, to work at the RHI level you should use the next layer of abstraction: command-lists.

The lowest level of graphics programming in Unreal is to make your own draw/compute calls on the Render Thread, using the global functions found at the bottom of `/Engine/Source/Runtime/RHI/Public/RHICommandList.h`. For example, you can create a bare-bones texture object with `RHICreateTexture(...)`.

One other global RHI function that I have actually used is `RHICreateSamplerState`, which makes a reference-counted handle to a sampler object. Higher abstractions don't seem to have functions for creating/destroying these (except using compile-time-constant parameters), so my impression is that they are not expensive to create.

RHI Command Lists

TL;DR: This is preferred to the previous layer (global function calls), and also gets used within RDG event callbacks. At worst their performance will be about the same as the global functions, and at best they will be better.

An `FRHICommandList` is a queue of draw calls and other state changes which will be executed asynchronously (meaning, at some point in the future within a frame or two). If your platform uses a multi-threaded rendering API like DirectX12 or Vulkan, those command-lists will be executed on multiple worker threads to minimize graphics driver overhead.

Along with the command list's member functions there are quite a few global functions offered by Unreal to accomplish common tasks. For example,

`DrawClearQuad()` sets up a draw call with an internal fragment shader that clears the current render target for you. You can also look inside the implementation of RDG global functions (see below) for examples of how to use command lists.

Usually you won't create command-lists yourself (though it's not hard, see the **Details** section), because they're given to you by various rendering hooks. They are decentralized objects, with no singleton manager and no standard lifetime, however you can only execute them once each so they tend to live inside one frame.

If you're already in the render thread then you can also grab a special singleton command list, of the child type `FRHICommandListImmediate`, which immediately executes each command rather than queueing them up. This type of command-list is also what you receive in your lambdas for `ENQUEUE_RENDER_COMMAND()`, presumably only for convenience since it's already a singleton.

Render Dependency Graph (RDG)

TL;DR: This is the layer you want to use, but within each RDG pass you will be given an RHI command list to work with.

An RDG (of the type `FRDGBuilder`) is a command-list on steroids. Each instance handles the allocation and destruction of temporary GPU resources *and* C++ objects, while executing the commands you give to it. They also optimize your graphics

commands in numerous ways, such as pooling memory allocations, reordering draw calls, and pruning unused commands. An entire Unreal frame is built with a small number of RDG instances. Every RDG must run in the render-thread (because it runs on an immediate-mode command list), so either create and execute your own graph inside `ENQUEUE_RENDER_COMMAND`, or use the one that's given to you in various render-thread callbacks covered in the next article.

The first half of RDG's existence is resource management. Any C++ objects you allocate with the functions `graph.Alloc[X]()` are guaranteed to live until the graph is done executing. Additionally any GPU resources you create within a graph will actually be created later, during execution, and then cleaned up immediately after they aren't needed anymore. The handle to these resources is a special RDG stub, which can be unwrapped into a proper RHI handle *only during execution*.

You can register existing RHI resources with the graph, called “external” resources, by calling `graph.RegisterExternal[X](...)`. This works even if the resource has already been registered with this graph by somebody else (despite engine code hinting otherwise).

On the other hand you can extract a temporary RDG resource into a permanent one with `graph.Queue[X]Extraction(...)`.

The second half of RDG is the ability to queue up GPU commands. All graphics operations in the graph are organized into *passes*. The simplest function you can call is `graph.AddPass(...)` which sets up render targets, issues resource memory barriers, and executes your own render-thread lambda. In order to know which resources your pass will use, `AddPass()` also takes a *shader parameter struct* (later articles will explain shaders) and assumes you use all of the RDG resources named in that struct. Listed below are all the ways you can extend a shader parameter struct to work with the RDG, though this might be confusing until you've written some Unreal shaders:

- Add the macro `RENDER_TARGET_BINDING_SLOTS()` to your parameter struct to give it a new field named `RenderTargets`. RDG will ensure that the settings you configure in this field (depth buffer, color buffers, etc) are bound right before

your pass lambda is executed.

- Replace the usual RHI texture and buffer parameters like `SHADER_PARAMETER_TEXTURE(...)` , with their RDG equivalents like `SHADER_PARAMETER_RDG_TEXTURE(...)` , to make RDG aware of the inputs and outputs of your pass. It's especially important for mutable resources to be seen by the RDG, in order to schedule those passes in the correct order.

There are many global functions provided by Unreal that perform common RDG tasks for you, usually named `Add[X]Pass(...)` . For example `AddCopyTexturePass(...)` , `AddClearRenderTargetPass(...)` , `AddSimpleMeshPass(...)` .

Demo

To illustrate use of some RHI commands, queued up from the game thread to the render thread, we will add a small demo Actor which manages a render target. That render target will be randomly cleared every so often to a new color.

I can't get this thing to loop on Medium, but you can right-click and press Loop!

Here is our actor's declaration (not including the Blueprint to display the texture on a plane):

```
#include "CoreMinimal.h"

UCLASS(BlueprintType)
class GOL_DEMO_API A_GOL_TestRHI : public AActor
{
    GENERATED_BODY()
public:

    UPROPERTY(BlueprintReadWrite, EditAnywhere, Transient)
    UTextureRenderTarget2D* TestOutput = nullptr;

    A_GOL_TestRHI() { PrimaryActorTick.bCanEverTick = true; }

    virtual void BeginPlay() override;
    virtual void Tick(float deltaSeconds) override;
};
```

And here is the definition:

```
#include "ClearQuad.h"
#include "Kismet/KismetRenderingLibrary.h"

void A_GOL_TestRHI::BeginPlay()
{
    Super::BeginPlay();

    const int Resolution = 16;
    TestOutput = NewObject<UTextureRenderTarget2D>(this);
    TestOutput->ClearColor = FLinearColor::Transparent;
    TestOutput->InitCustomFormat(Resolution, Resolution,
                                PF_R8G8B8A8, true);
}

void A_GOL_TestRHI::Tick(float deltaSeconds)
{
    Super::Tick(deltaSeconds);

    auto* resource = TestOutput->GetResource();
    auto* rhi = (resource == nullptr) ? nullptr : resource->GetTexture2DRHI();

    auto randF = []{ return FMath::FRandRange(0.0f, 1.0f); };
};
```

```

    if (rhi && randF() < 0.09f)
    {
        FLinearColor clearColor{ randF(), randF(), randF(), 1 };

        ENQUEUE_RENDER_COMMAND(RandomizedClear)([rhi, clearColor](FRHICommandList& Cmds)
        {
            //NOTE: Clearing textures is a lot easier to do with RDG;
            //      this is just an illustrative example of RHI.

            //Start using the texture as a render target.
            Cmds.Transition(FRHITransitionInfo{ rhi, ERHIAccess::Unknown, ERHIAccess::RenderTarget,
            Cmds.BeginRenderPass(FRHIRenderPassInfo{ rhi, ERenderTargetActions::Discard,
            TEXT("RandomizedClear"));

            //Clear it to the random color.
            DrawClearQuad(
                Cmds, true, clearColor,
                //No depth/stencil.
                false, 0, false, 0
            );

            //Clean up.
            Cmds.EndRenderPass();
            Cmds.Transition(FRHITransitionInfo{ rhi, ERHIAccess::RTV, ERHIAccess::RenderTarget,
            });
        });
    }
}

```

Code

The demo RHI actor is added to the repo in the *part/2* branch.

[heyx3/UnrealExtendedGraphicsProgrammingDemo at part/2](#)

Details

While RDG's `AddPass()` takes a shader parameter struct, it does not upload that struct to the shader for you. That's because `AddPass()` calls aren't tied to any particular shader — it doesn't know where to send them. However most of the global RDG helper functions take shaders, such as `AddDrawScreenPass()`, and they will also upload your parameter struct for you. The basic details of parameter uploading are covered in the fourth article.

Under the hood, global RHI functions will call virtual member functions on the

abstract classes `FDynamicRHI` and `IRHICommandContext`. These contain the actual calls to Vulkan, DirectX, etc. On simple platforms (no multithreaded rendering) one class will inherit from both types; on more cutting-edge platforms the context is a separate object and not a singleton.

The standard command-list type is `FRHICommandList`, but there is a compute-only subset called `FRHIComputeCommandList`.

Anybody can construct command lists, queue up commands (synchronously, no multithreading within a single command-list), then dispatch it with

`FinishRecording()`. After `FinishRecording()` you should not touch the object anymore. If you fail to call `FinishRecording()`, an internal assert will trip in the destructor to tell you about your bug.

Ostensibly you can create/use a command-list on any thread, especially given the existence of the helper struct `FRHIAsyncCommandList`, however Unreal crashed when I tried to use a command-list from the game thread. Maybe it's a bug with `DrawClearQuad()`, or maybe Epic gave up on async command lists?

RDG's, like command-lists, can be created and used manually on the render-thread. Construct one with an immediate-command-list, for example

```
FRDGBuilder graph{ FRHICommandListImmediate::Get() };
```

then queue up passes with it and call `graph.Execute()` to finish. In practice you won't need to do this often; more individual RDG's means less opportunity for pooling resources anyway!

[Unreal Engine 5](#)[Graphics Programming](#)[Shaders](#)[Technical Art](#)

[Edit profile](#)

Written by William Mishra-Manning

59 followers · 5 following

Graphics Programming, Procedural Generation

Responses (1)



William Mishra-Manning

What are your thoughts?



Tan Yu Tao

Oct 14, 2025 (edited)



Hi William, really enjoying the entire series so far!

I was wondering, if you have more details on the sceneproxy functions in Unreal Engine.

I understand that the proxies are the renderthread version of the gamethread object, storing the data that is...

[more](#)



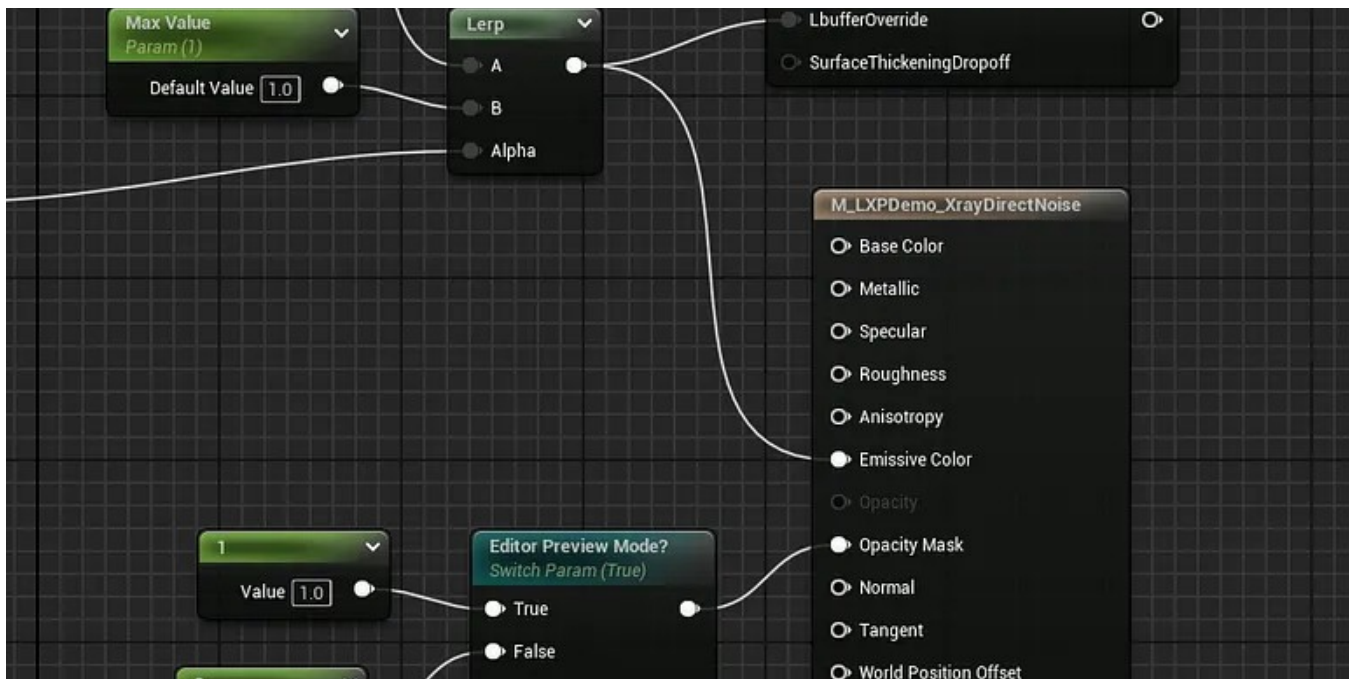
1



1 reply

[Reply](#)

More from William Mishra-Manning

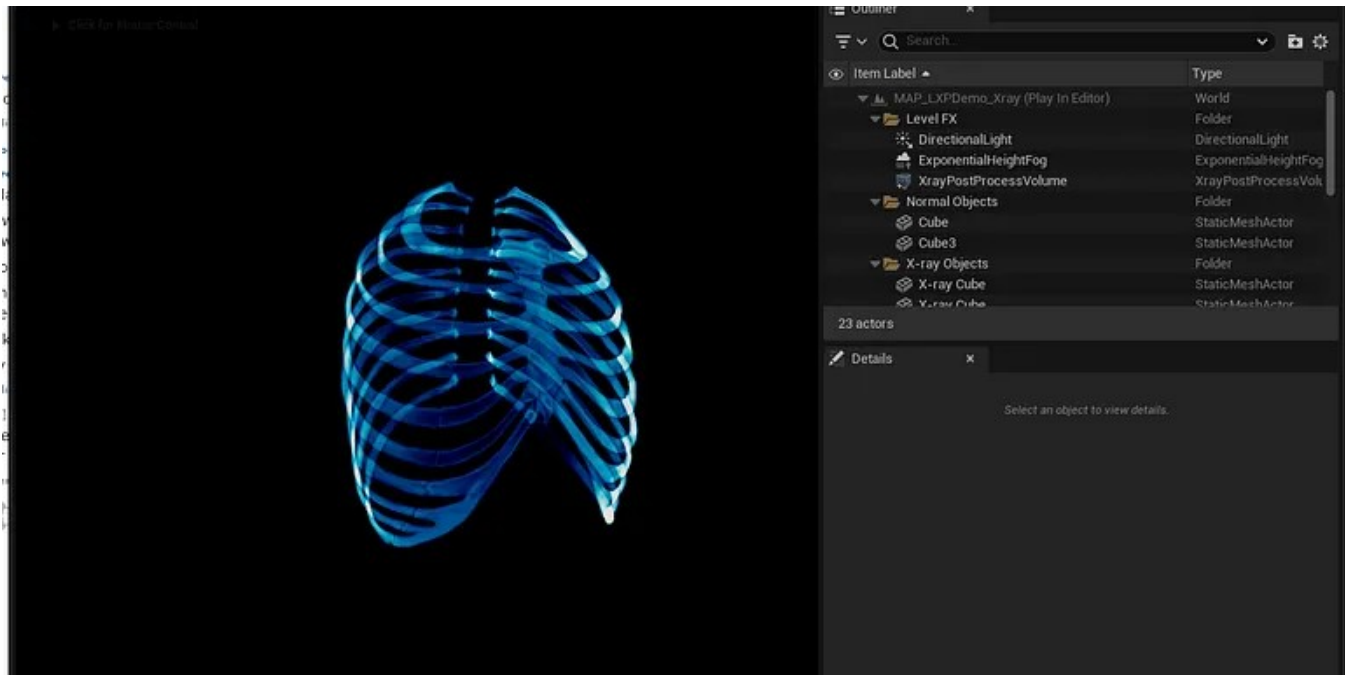


 William Mishra-Manning

Advanced Graphics Programming in Unreal, part 6

This is a seven-part article series.

Jun 25, 2025  1  2





William Mishra-Manning

Advanced Graphics Programming in Unreal, part 1

This is a seven-part article series.



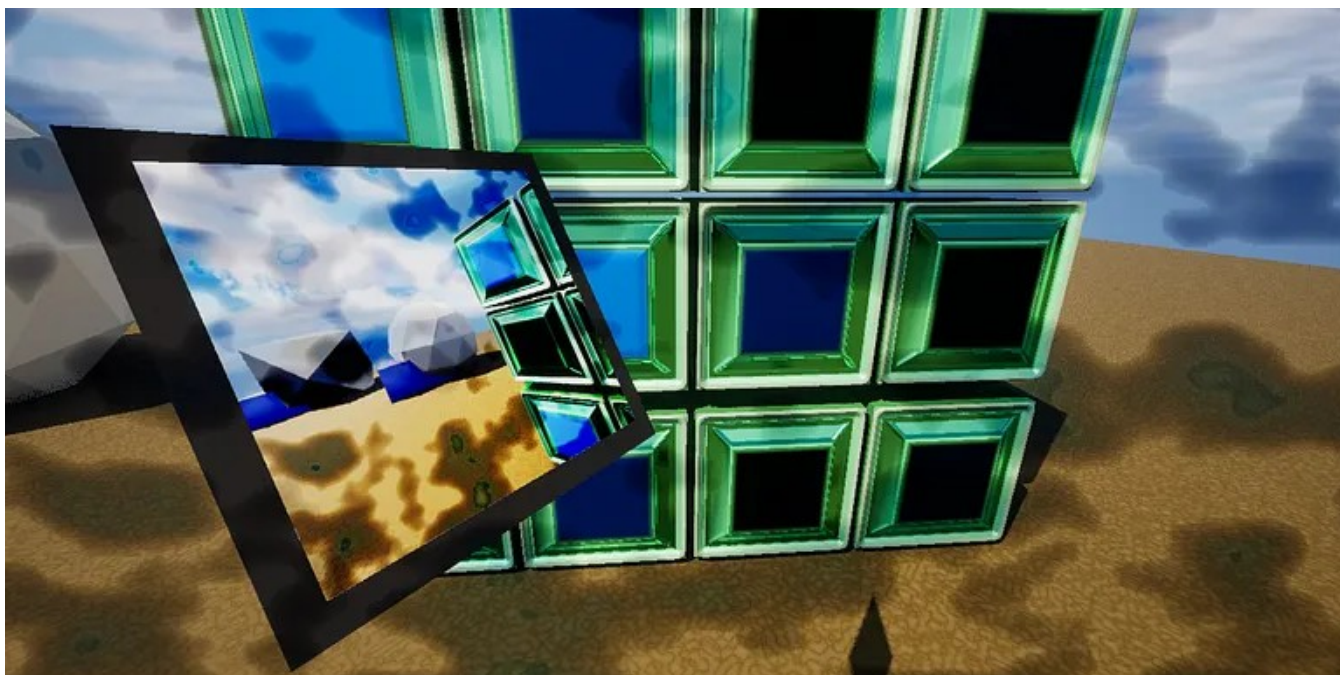
William Mishra-Manning

Advanced Graphics Programming in Unreal, part 3

This is a seven-part article series.

Jun 24, 2025 6





William Mishra-Manning

Advanced Graphics Programming in Unreal, part 4

This is a seven-part article series.

Jun 25, 2025 🖱 18



See all from William Mishra-Manning

Recommended from Medium



Seulgie Han

C++ Move Semantics: From Copy to Efficient Resource Transfer

Source & Purpose: Based on Udacity C++ Nanodegree — Course3: Memory Management, this post is intended to help consolidate learning about...

Dec 13, 2025



 Kaloy Muñoz

Why CD Projekt Red Might Be the Last Prestige RPG Developer That Still Gets It

While the gaming industry shifts toward live-service models and diluted storytelling, CD Projekt Red continues to pursue ambitious...

Oct 23, 2025  6

 Kastuvpokharel

QNetworkInformation (C++) vs NetworkInformation (QML): performance, efficiency, security and what...

If you want to know what QNetworkInformation is read this short article first, as this will be the perquisite for this article...

Dec 15, 2025



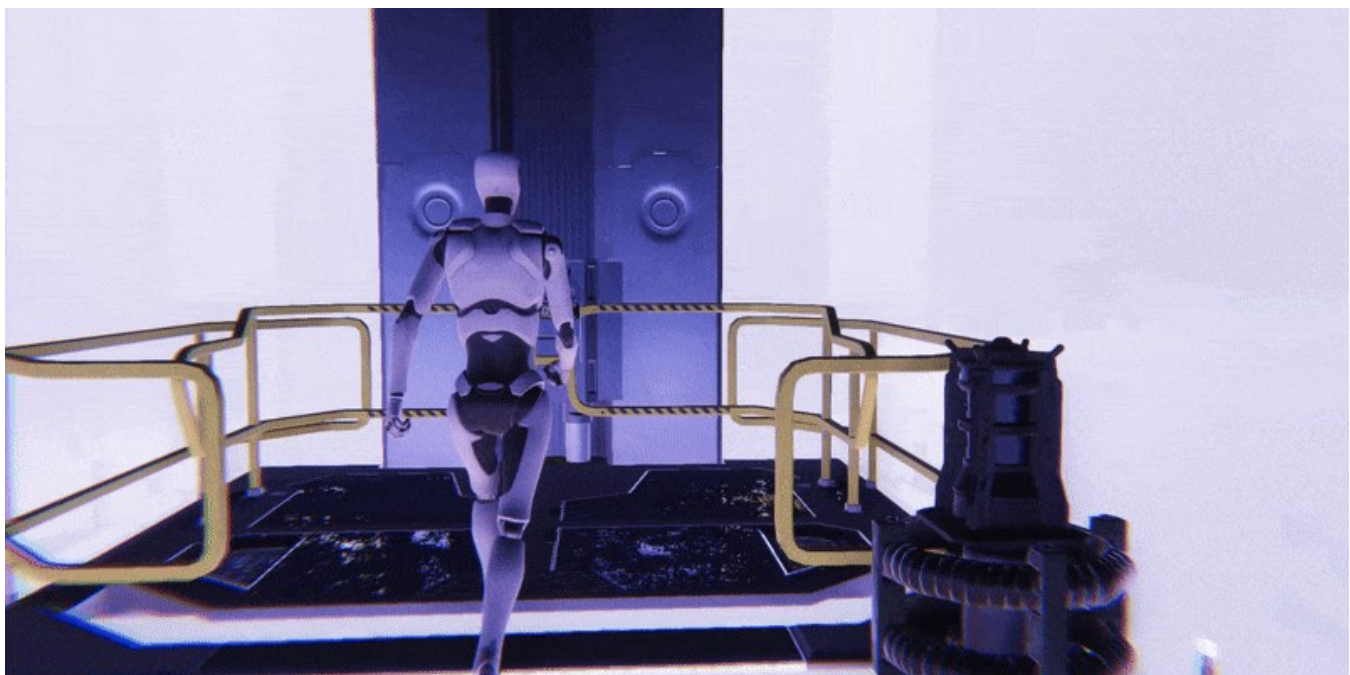


 Tutorial Man

Build & Deploy a Mobile Multiplayer Game with Unity — From Scratch to Android/iOS

Read till last clap follow share

Nov 7, 2025  1



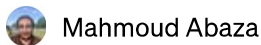
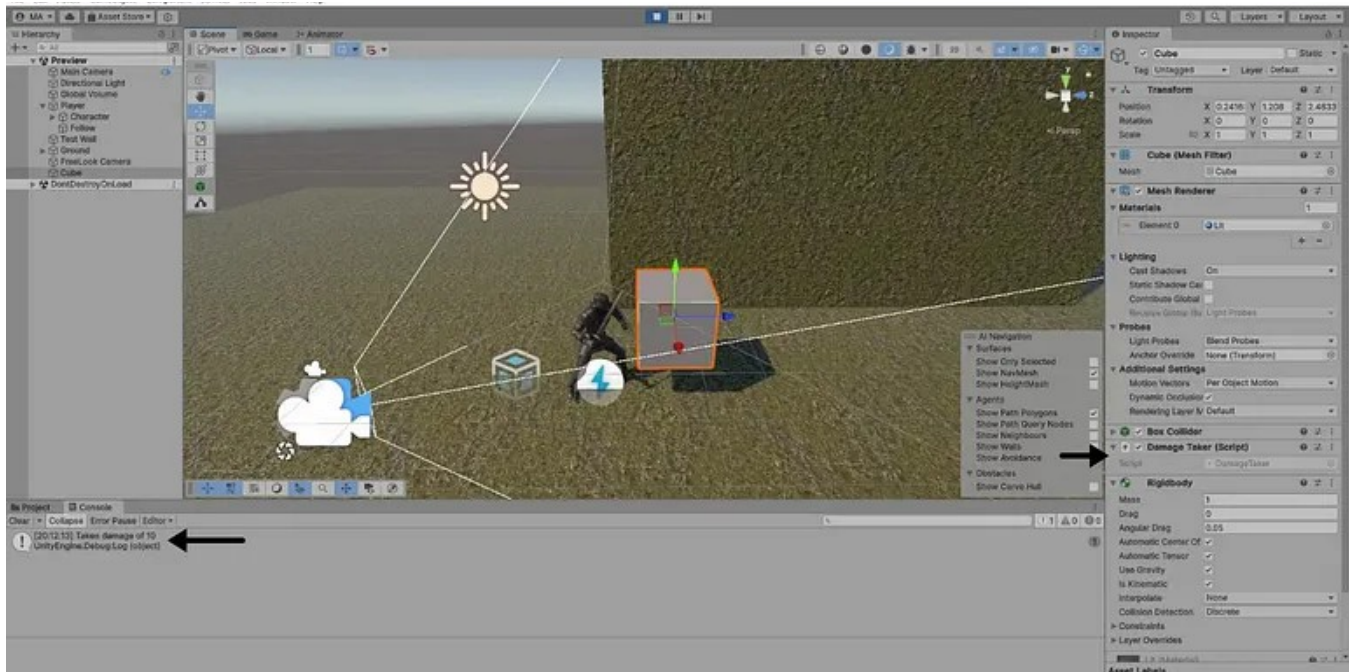
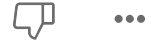


Jared Amlin

Elevator Mechanics in Unity

Elevators are a great addition to interior level design and let the player move between floors. Let's take a look at one way to make a...

Nov 7, 2025 5

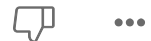


Mahmoud Abaza

Building an RPG in Unity Part 3: Setting up camera and fighting system

In the previous chapter we explored how to add characters, animations and move them around into our game, but we have not given the camera...

Dec 16, 2025 9

[See more recommendations](#)